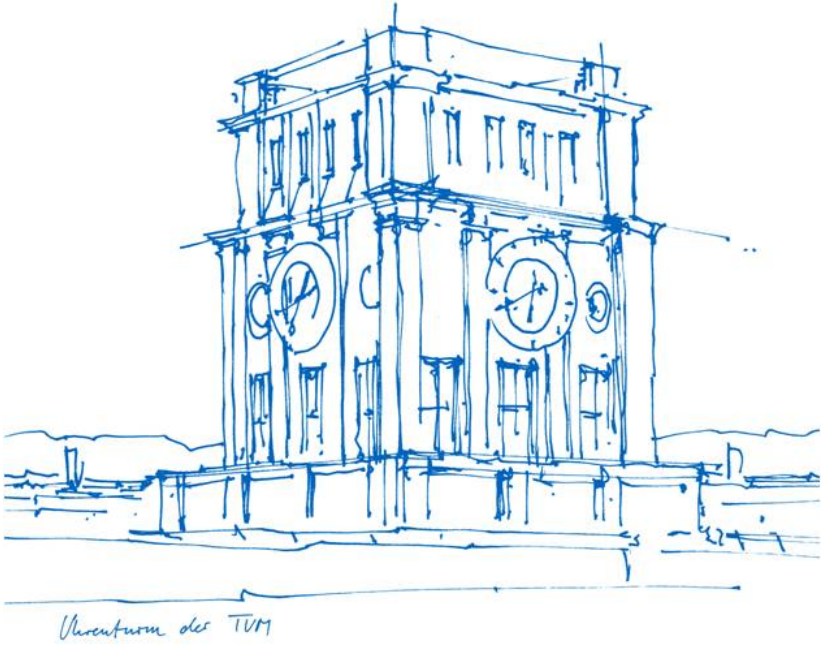
Persistent Memory I/O Primitives

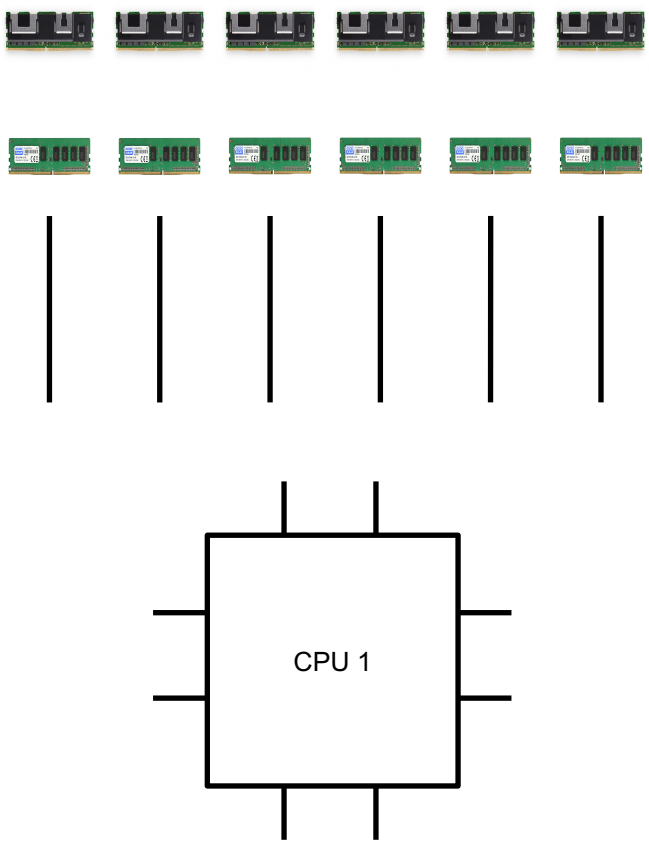
Alexander van Renen, Lukas Vogel, Viktor Leis[1]

Thomas Neumann, Alfons Kemper



[1] now at Friedrich-Schiller-Universität Jena

Evaluation System Architecture



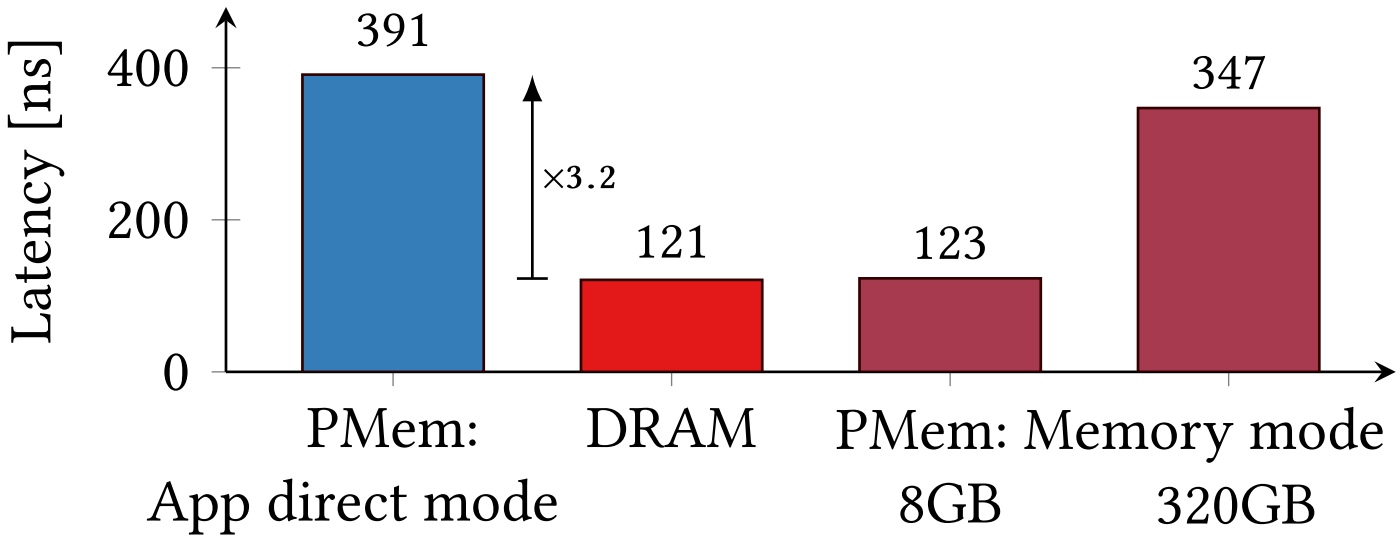
6 Intel Optane DC DIMMs with 128GB each = 768GB

6 regular DDR4 DRAM DIMMs 32GB each = 192GB

6 channels

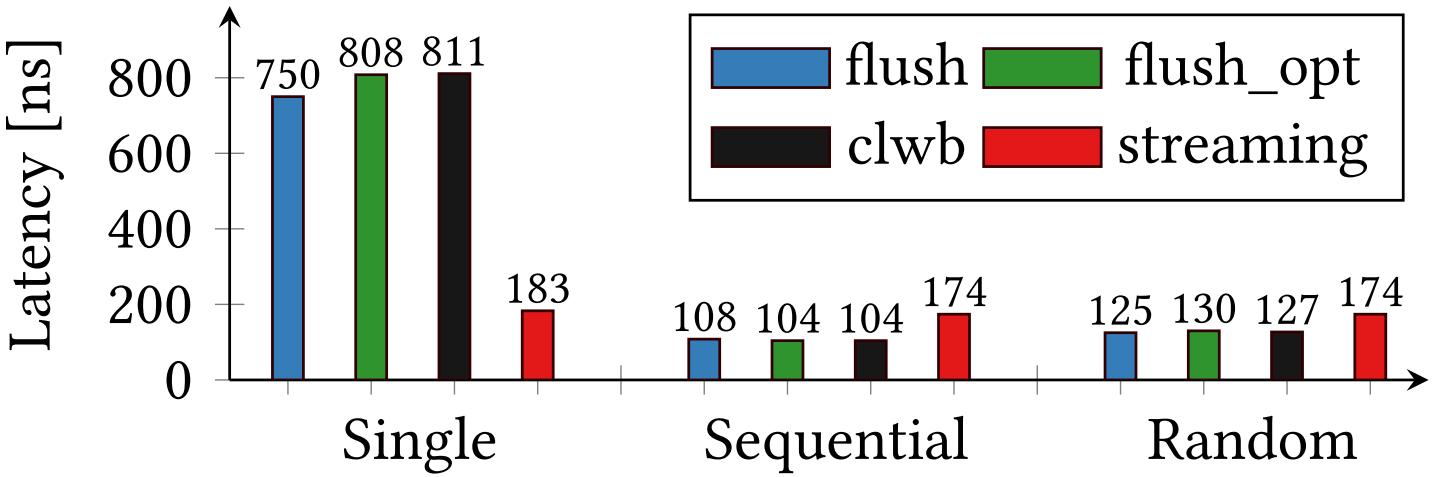
Intel's second generation Xeon scalable processors (Cascade Lake) with 24 physical cores @2.2GHz

Read Latency (Dependent random 8 byte reads)



- **Rule 1**: Prefer reading from DRAM, if possible
- In Memory Mode: The DRAM (“L4”) cache can hide the PMem latency

Persistent Write Latency (8 byte writes, ordered with SFENCE)



- **Rule 2**: Avoid writing the same cache line (or use streaming)
- **Rule 3**: Otherwise, use `clwb` (Current CPUs emulate `clwb` with `flush_opt`)

- (Note: SFENCE is always used to ensure ordering, except for streaming)

Logging into an Array

```
char[4] data;
unsigned size;

void PushBack(char c)
{
  data[size] = c;
  size++;
}
```

All possible states on PMem

size = 1

A	NUL	NUL	NUL

PushBack('B');

size = 1

A	NUL	NUL	NUL

size = 2

A	NUL	NUL	NUL

size = 1

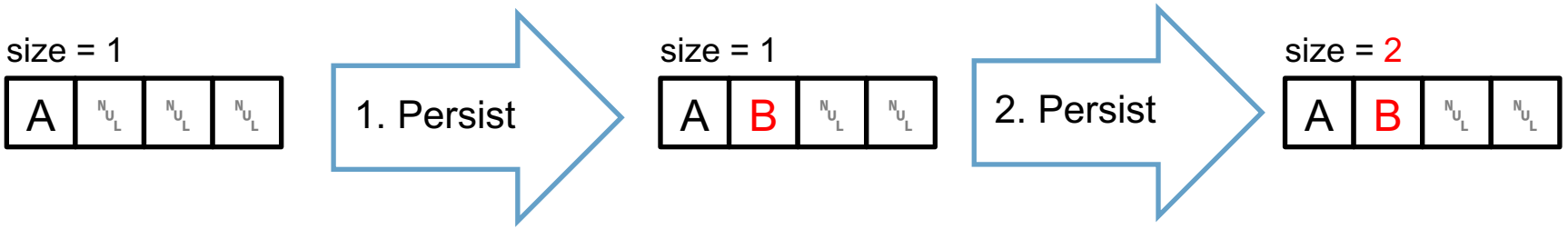
A	B	NUL	NUL

size = 2

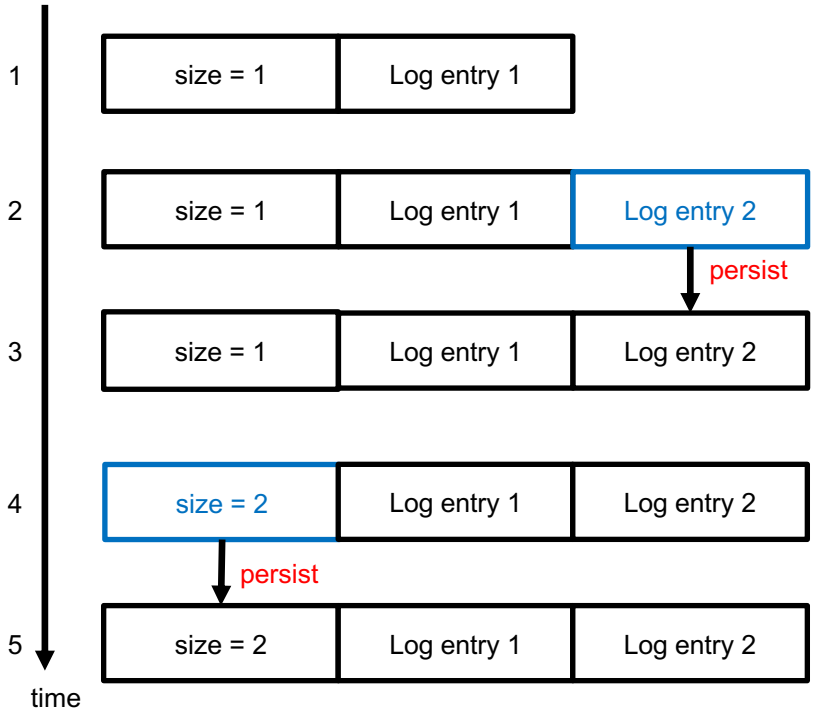
A	B	NUL	NUL

Logging into an Array

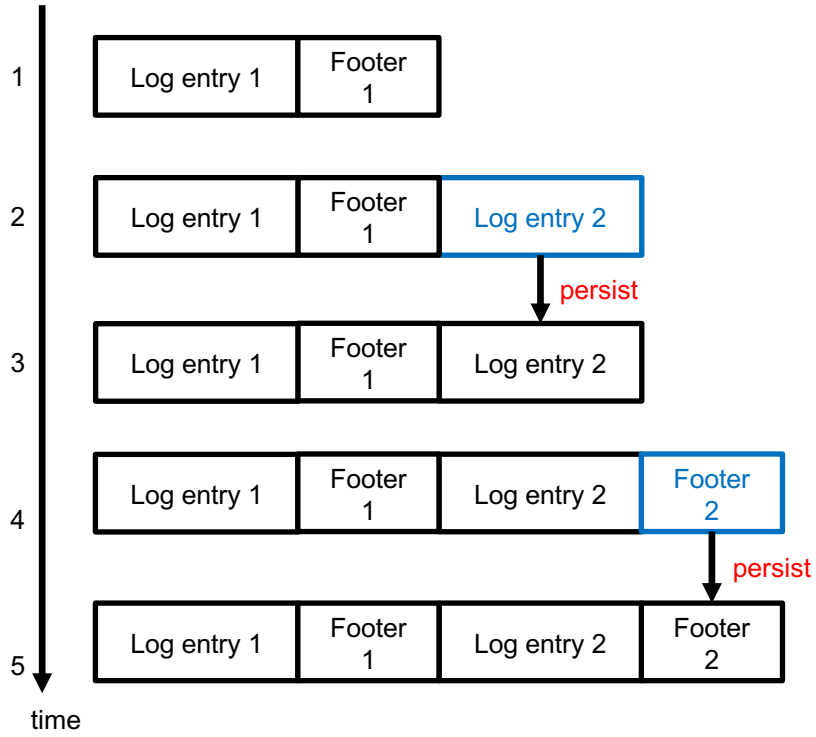
```
void Persist(void *addr)
{
   _mm_clwb(addr);
   _mm_sfence();
}

void PushBack(char c)
{
   data[size] = c;
   Persist(&data[size]);
   size++;
   Persist(&size);
}
```



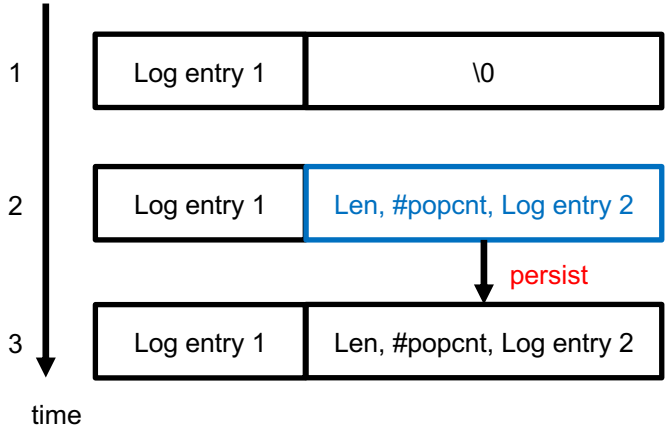
Logging Algorithm 1: Header / libpmemlog



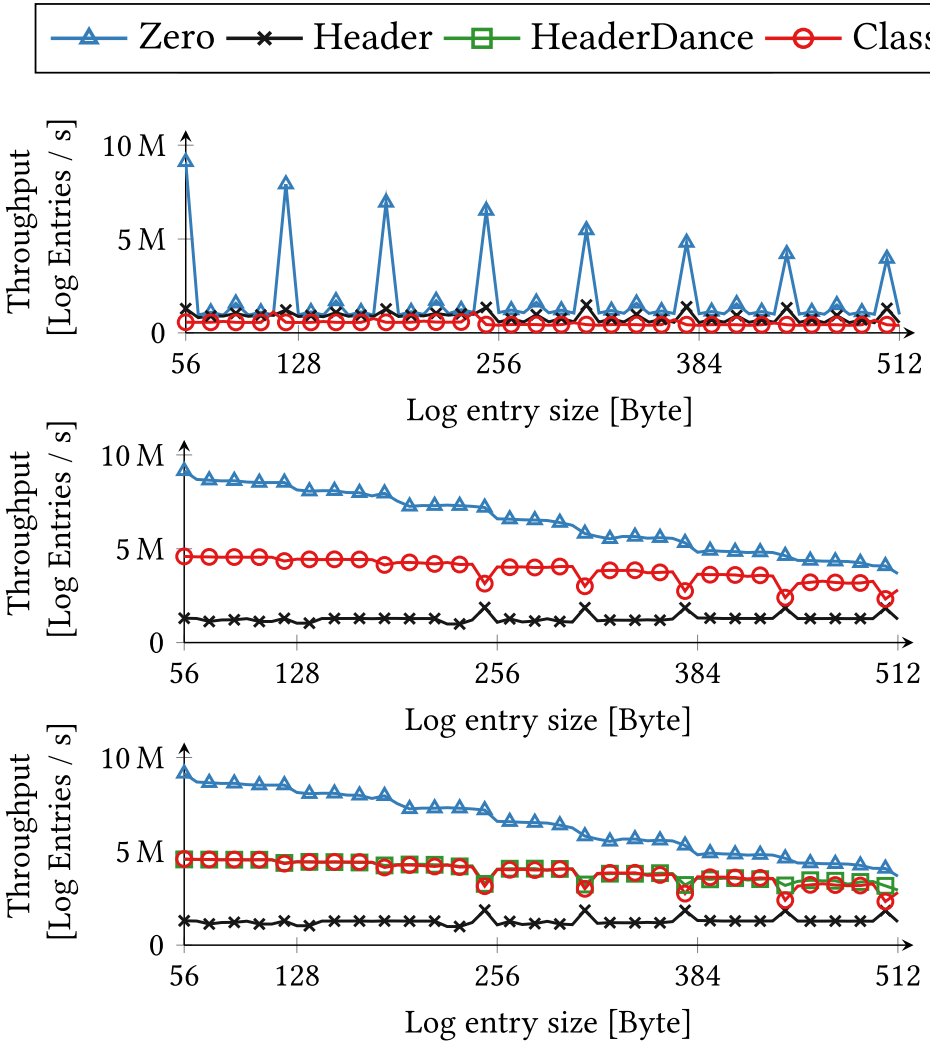
Logging Algorithm 2: Classic / Footer



Logging Algorithm 3: Zero

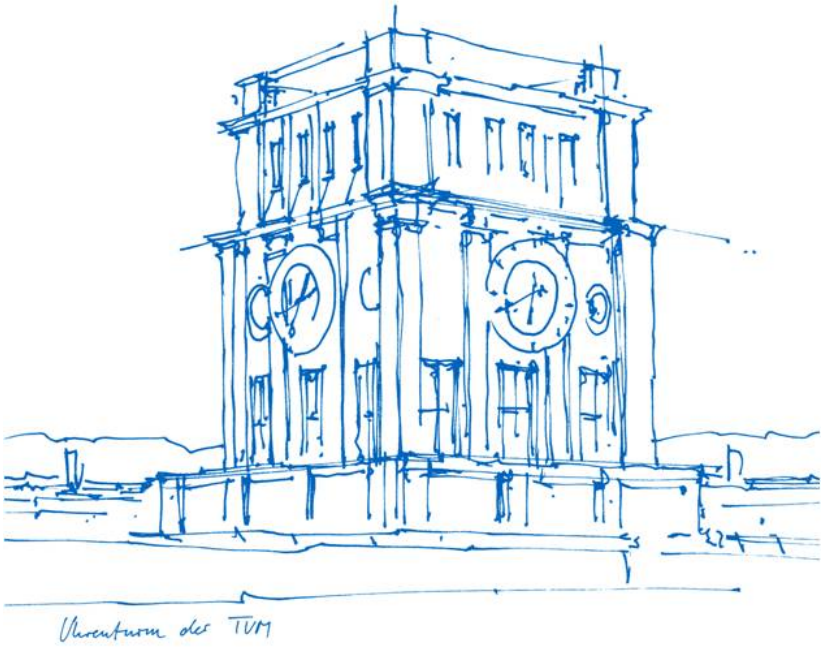


Logging Evaluation

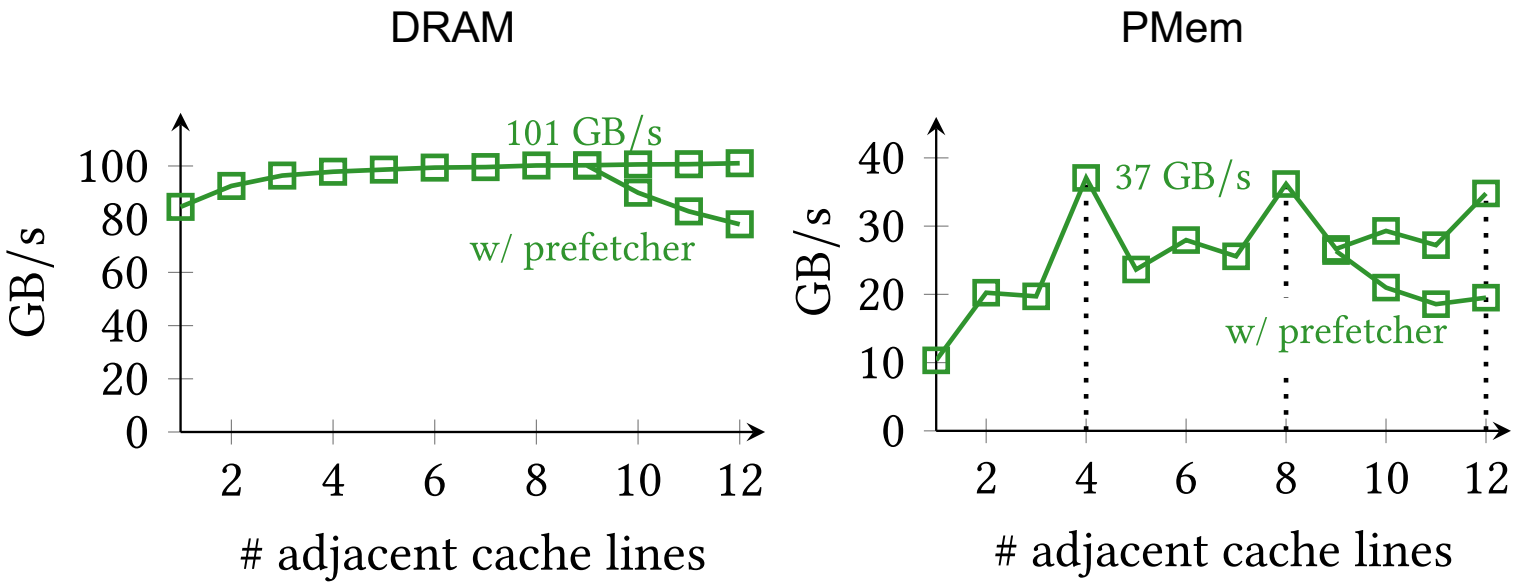


- In all approaches multiple writes are hitting the same cache line (**Rule 2**)
  - ➢ Use alignment
- Size field in header has to be inline
  - ➢ Use streaming or multiple size fields
- **Rule 4**: Minimize number of persistency barriers

Part 2: Bandwidth

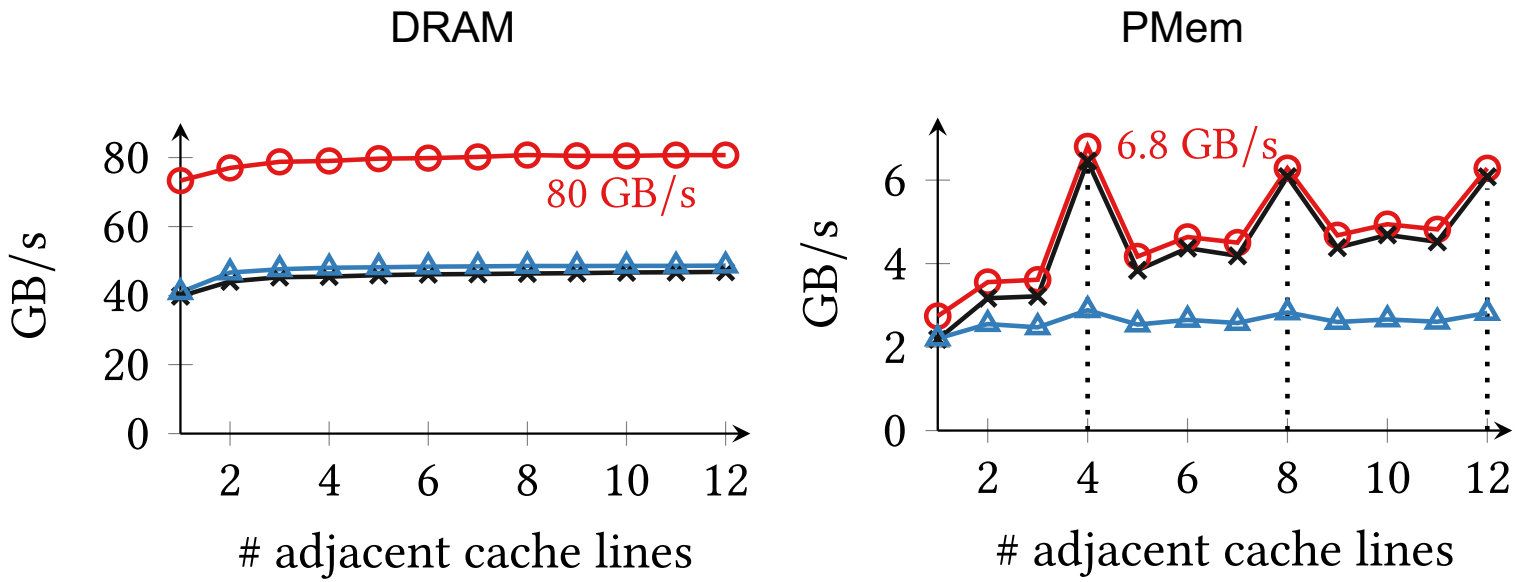


Read Throughput (24 threads)



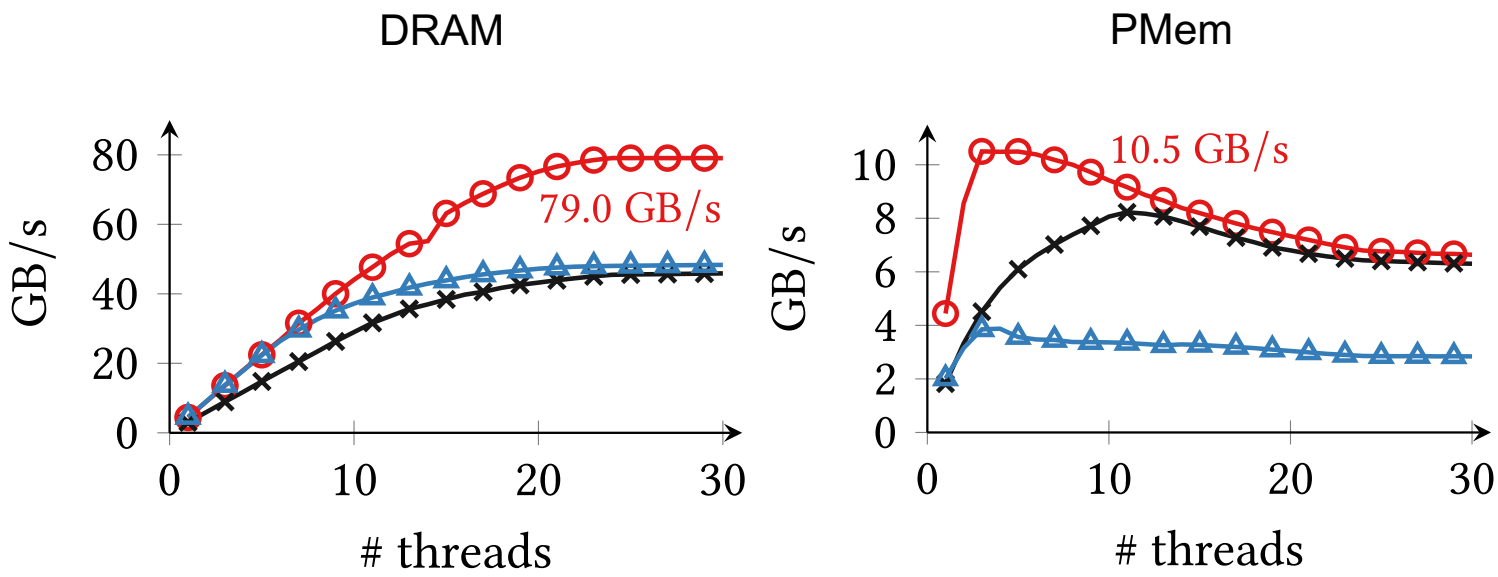
- **Rule 5**: Algorithms should optimize for 256 byte blocks (rather than cache lines / 64 byte)
- 2.7x throughput difference

Write Throughput (24 threads)



- **Rule 5**: Algorithms should optimize for 256 byte blocks (rather than cache lines / 64 byte)
- 11.7x throughput difference (7.5 in optimal)

Write Throughput (4 adjacent cache lines)



- **Rule 6**: Explicitly order writes to PMem with `clwb`

- (Note: Not a problem for reads)

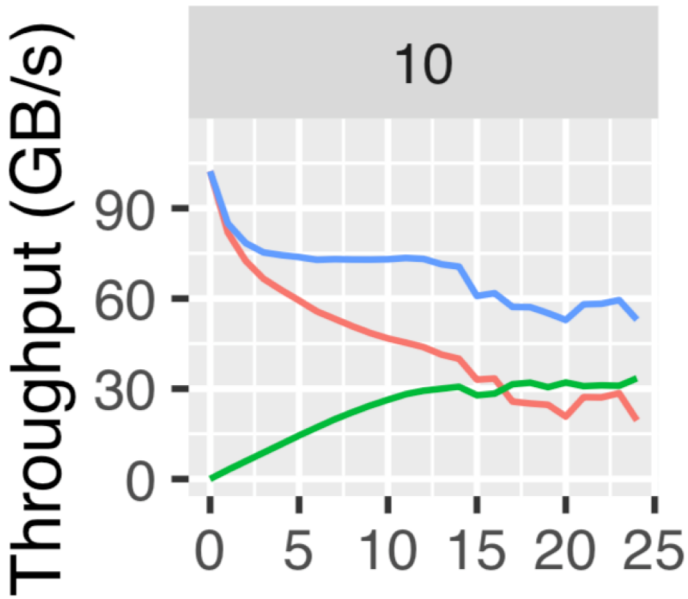
Summary

	Latency		Bandwidth	
	Read	Write	Read	Write
DRAM	121ns	N/A	101GB/s	79.0GB/s
PMem	391ns	104ns	37GB/s	10.5GB/s
Factor	3.2	N/A	2.7	7.5

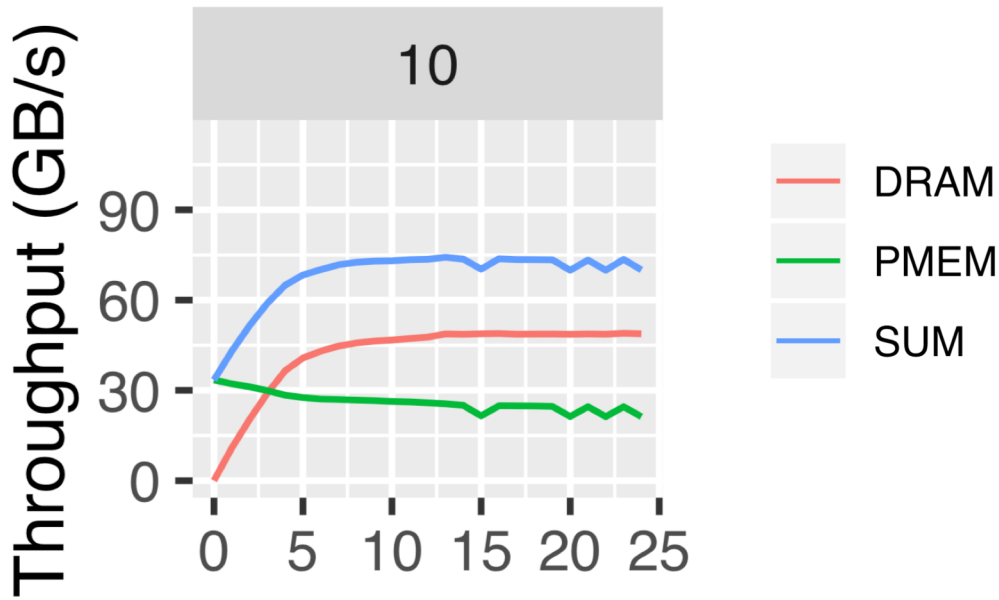
- **Rule 1**: Prefer reading from DRAM, if possible (~3x slower if not cached)
- **Rule 2**: Avoid writing the same cache line (or use streaming)
- **Rule 3**: Use `clwb` for persisting (Current CPUs emulate `clwb` with `flush_opt`)
- **Rule 4**: Minimize number of persistency barriers
- **Rule 5**: Algorithms should optimize for 256 byte blocks (rather than cache lines / 64 byte)
- **Rule 6**: Explicitly order writes to PMem with `clwb`, even if not needed for persistency

Bandwidth Congestion

Sequential reads from DRAM and PMem concurrently



10 DRAM threads, and varying PMem thread count on x-axis.



10 PMem threads, and varying DRAM thread count on x-axis.

Managing Non-Volatile Memory in Database Systems

-- Originally presented at SIGMOD 2018

Alexander van Renen, Viktor Leis,
Alfons Kemper, Thomas Neumann

Takushi Hashida, Kazuichi Oe, Yoshiyasu Doi,
Lilian Harada, Mitsuru Sato

Terminology, Assumptions, and Background

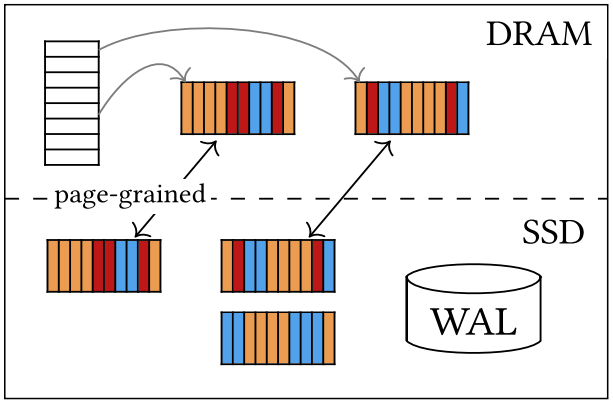
- For this talk: NVM (PMem, NVRAM, SCM, NVMM)
- NVM assumptions:
  - NVM is byte addressible
  - NVM has a higher access latency than DRAM
  - NVM has lower cost/GB than DRAM
  - NVM has higher capacity than DRAM

- Sources:
  - Paper: https://db.in.tum.de/~leis/papers/nvm.pdf
  - Video: https://youtu.be/6RRe_cmDl0U

Database Architectures

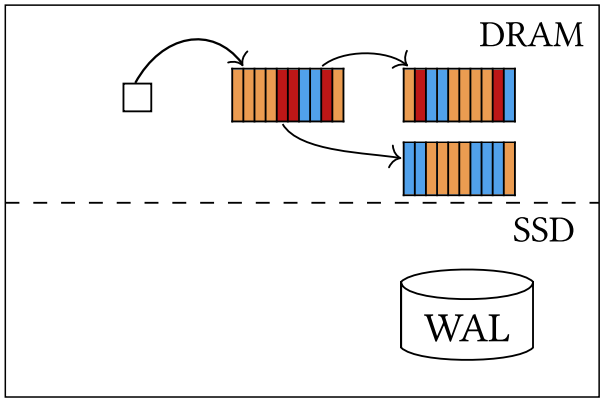
Disk-based DBs

- Primary data location on disk
- Loaded to DRAM for processing
- Logging to SSD



Main Memory DBs

- Primary data location in DRAM
- Snapshots written to SSD
- Logging to SSD



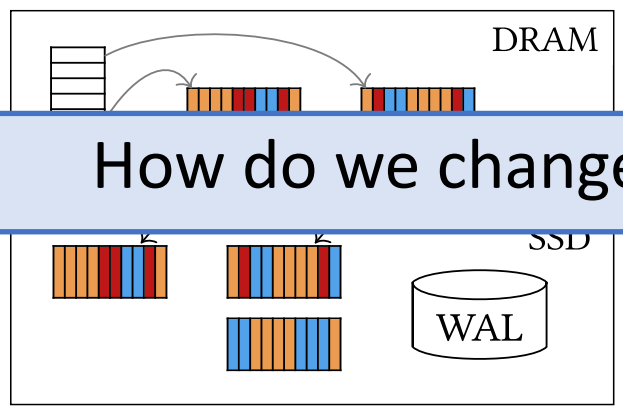
Database Architectures

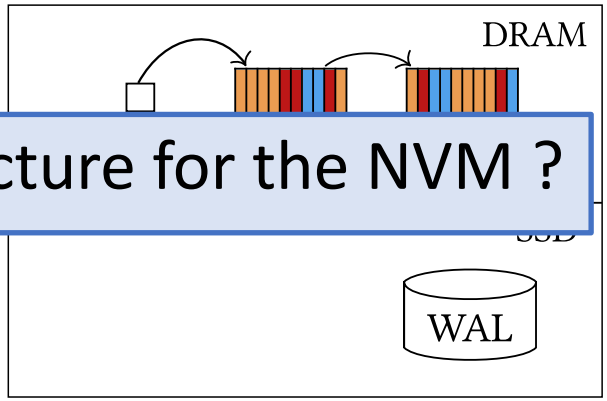
Disk-based DBs

- Primary data location on disk
- Loaded to DRAM for processing
- Logging to SSD

Main Memory DBs

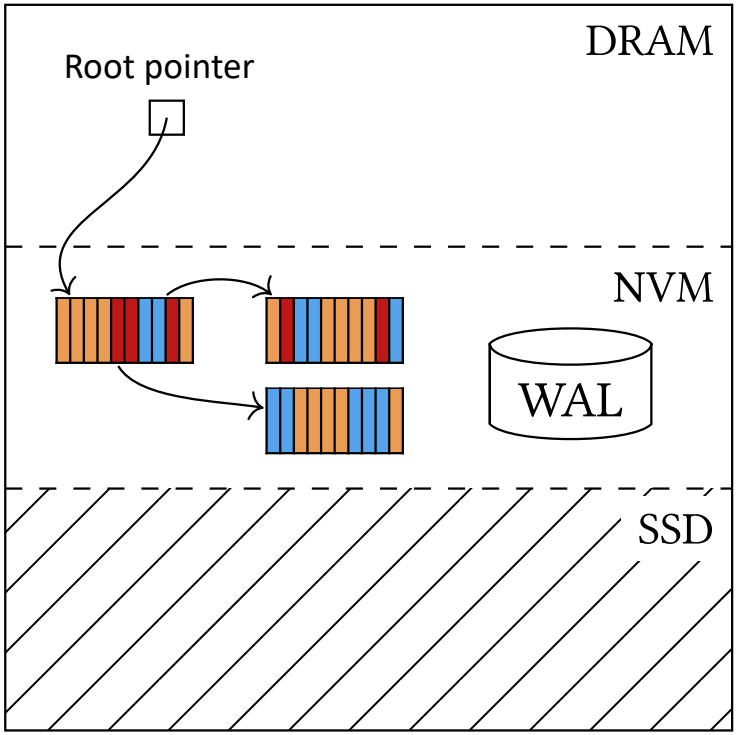
- Primary data location in DRAM
- Snapshots written to SSD
- Logging to SSD





How do we change dbs architecture for the NVM ?

NVM-direct Approach



In-place updates

- Requires failure atomicity
- High NVM latency
- No DRAM
- No SSD

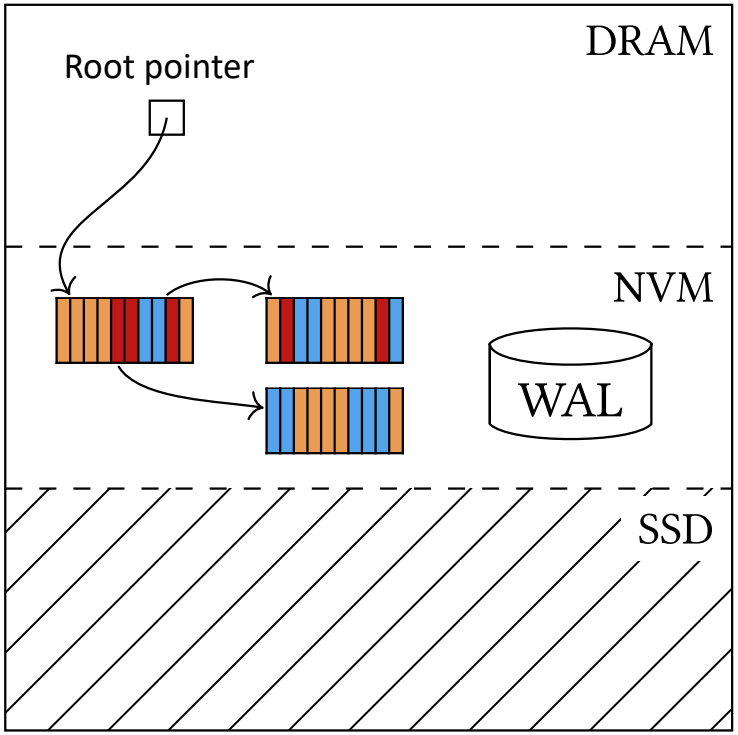
CDDS-Tree [VLDB 2015], NV-Tree [FAST 2015], wB-Tree [VLDB 2015], FP-Tree [SIGMOD 2016], WO[A]RT/ART+CoW [FAST 2017], HiKV [USENIX ATC 2017], Bz-Tree [VLDB 2018], BDCC+NVM [ICDE 2018], SAP Hana for NVM [VLDB 2017]

NVM-direct Data Structures

	Failure Atomicity	Concurrency	NVM Utilization	Placement
CDDS-Tree [61]	· versioning + atomics · shadowing (split,merge)	· only multiple-reader, single-writer model	· re-using dead versions	NVM-only
NV-Tree [68]	· append-only + atomics · reconstruction	· latches on modification · less locks due to append-only character	· unsorted leafs · cache-optimized format · selective consistency	NVM-only
wB-Tree [20]	· indirection + invalidation · redo-only logging (split,merge)	N/A	· unsorted nodes · binary search by sorted slot array	NVM-only
FP-Tree [52]	· indirection + invalidation · lightweight logging	· hardware-based + fine-grained locks	· reduced footprint · unsorted leafs · hashing · group allocation	NVM + DRAM
WO[A]RT/ ART+CoW [39]	· indirection + atomics · shadowing + atomics	N/A	· avoid key comparisons and tree rebalancing	NVM-only
HiKV [66]	· shadowing + atomics	· hardware-based for tree · locking within hash partitions	· avoids costly structure operations	NVM + DRAM
Bz-Tree [16]	· persistent multi-word CaS operation (PMwCAS)	· PMwCAS + spinning · optimistic protocol (insert)	· near-instantaneous recovery	NVM-only
BDCC+ NVM [28]	· undo logging through PMDK	N/A	· clustering instead of additional indexes · unsorted blocks · instant recovery	NVM + SSD

[Data Management on Non-Volatile Memory: A Perspective @Datenbank-Spektrum 18]

NVM-direct Approach

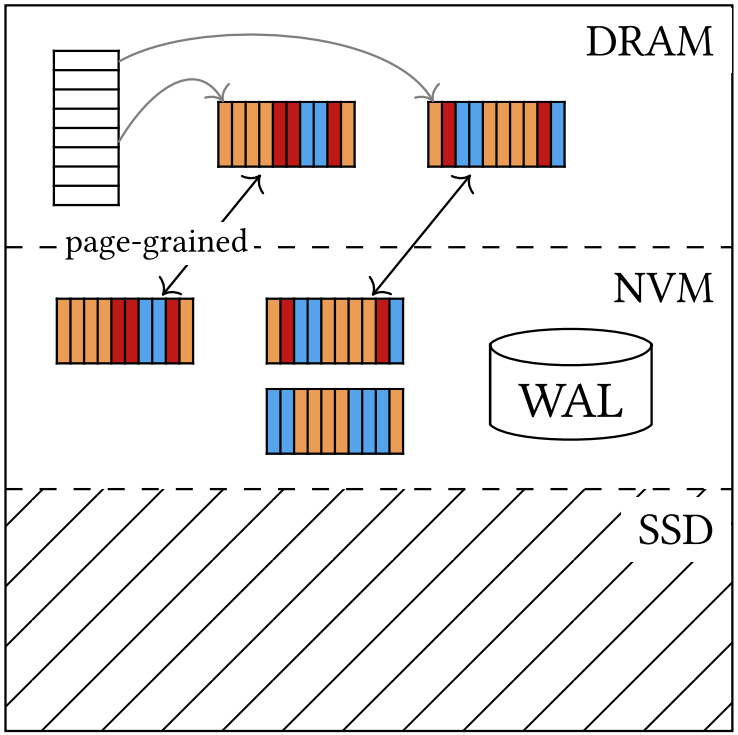


In-place updates

- Requires failure atomicity
- High NVM latency
- No DRAM
- No SSD

CDDS-Tree [VLDB 2015], NV-Tree [FAST 2015], wB-Tree [VLDB 2015], FP-Tree [SIGMOD 2016], WO[A]RT/ART+CoW [FAST 2017], HiKV [USENIX ATC 2017], Bz-Tree [VLDB 2018], BDCC+NVM [ICDE 2018], SAP Hana for NVM [VLDB 2017]

Buffered Approach

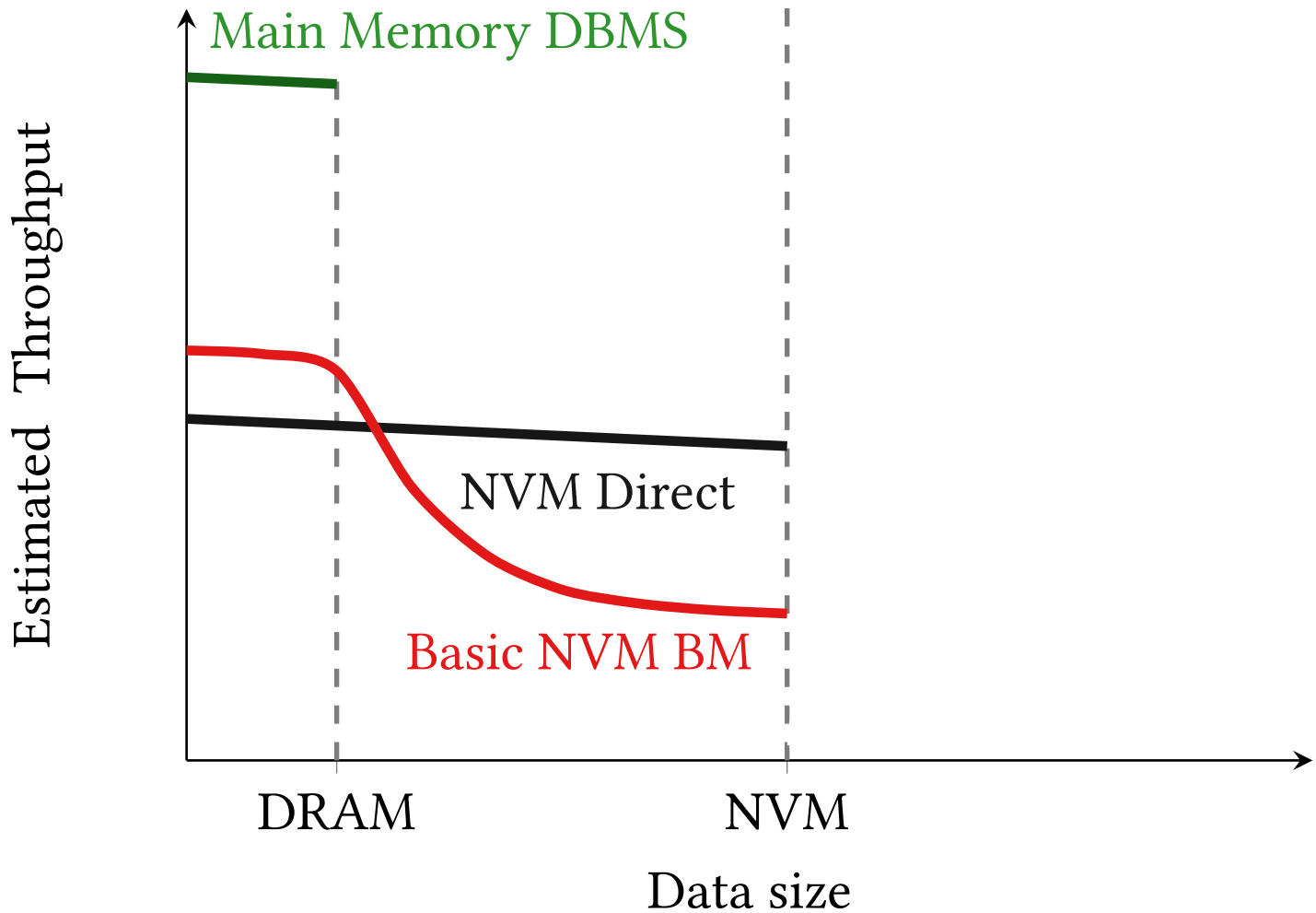


Out-of-place updates

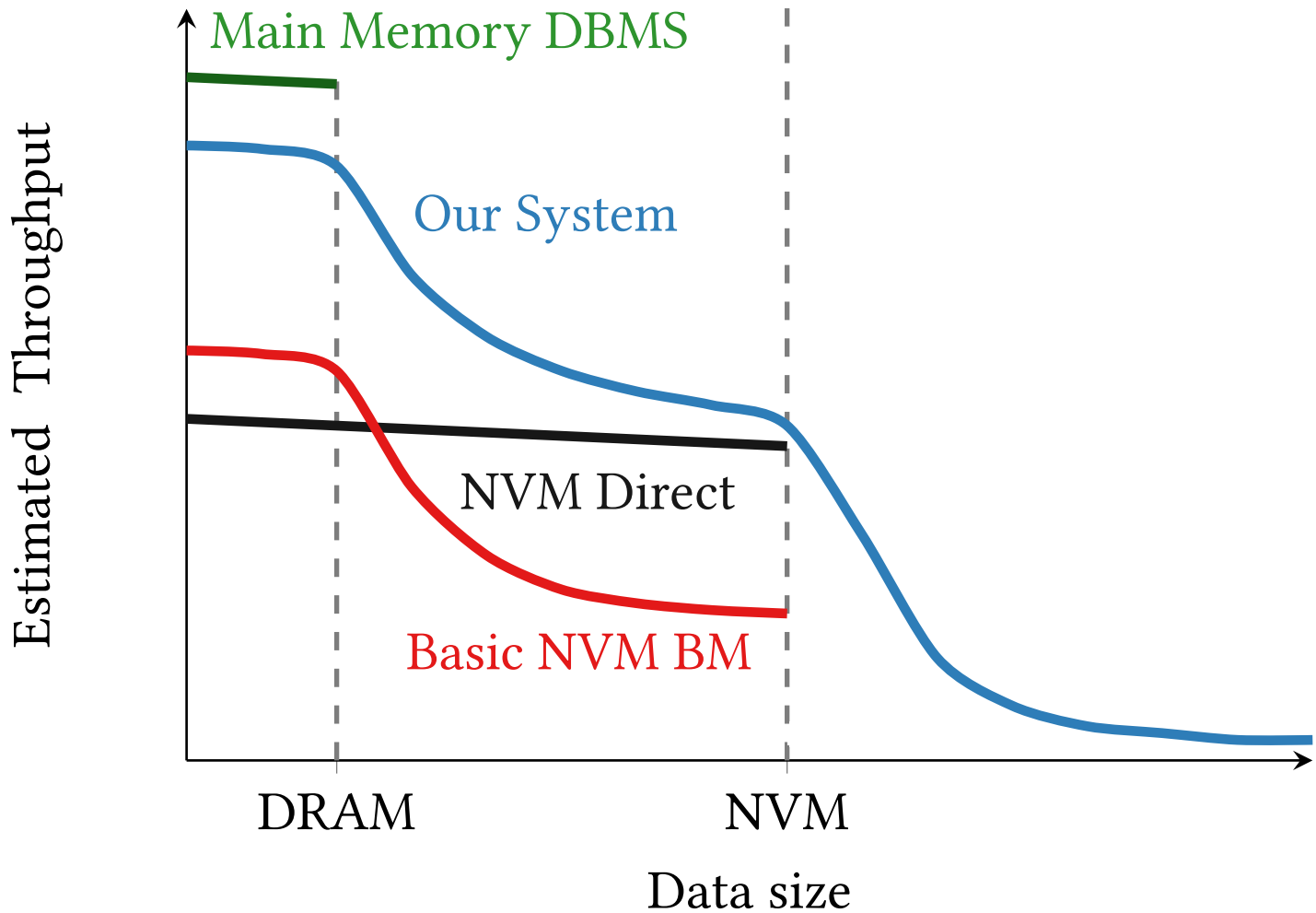
- No byte-addressability
- No SSD

FOEDUS [SIGMOD 2015], SAP Hana for NVM [VLDB 2017]

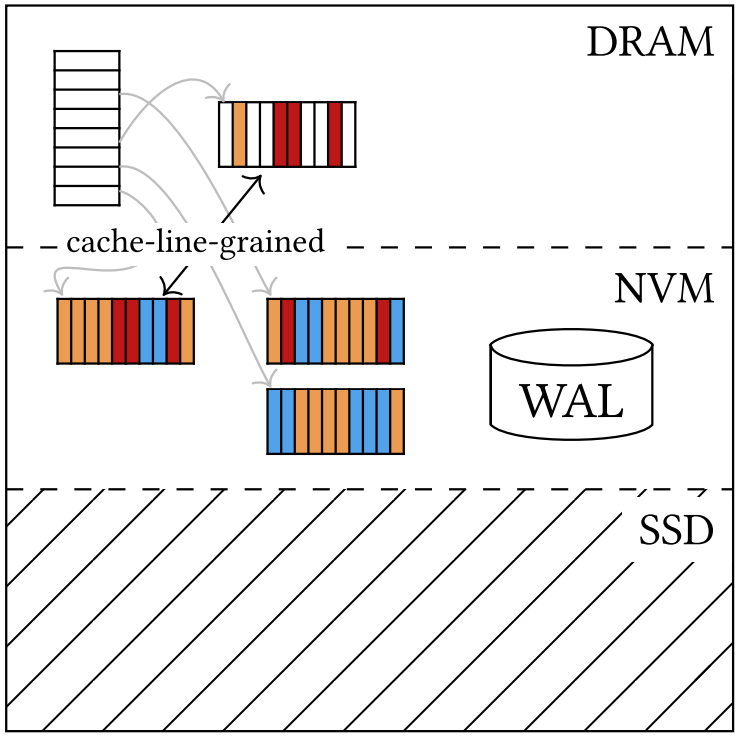
State of the Art



The Ideal System: “Dream Chart”

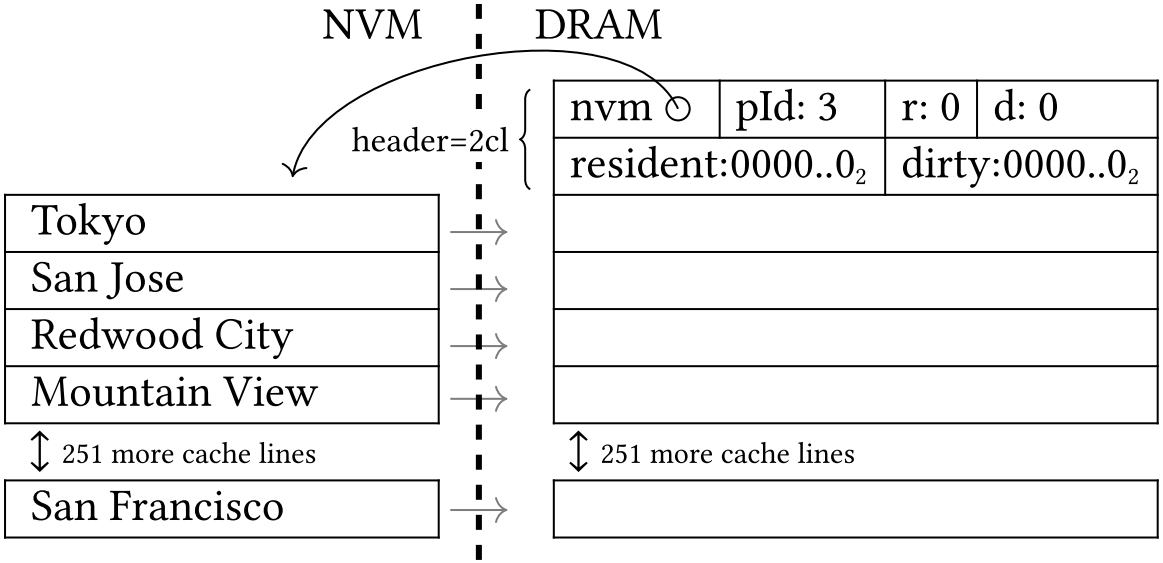


1. Cache-Line-Grained Loading



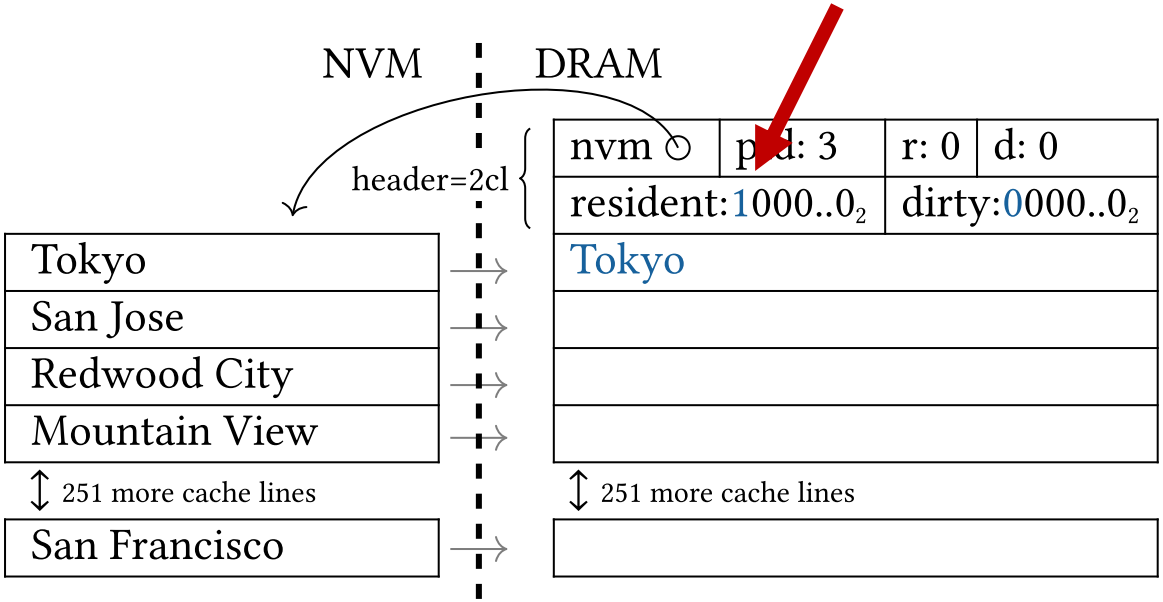
We transfer individual cache lines (64Byte) instead of entire pages (16KB) between DRAM and NVM.

1. Cache-Line-Grained Loading



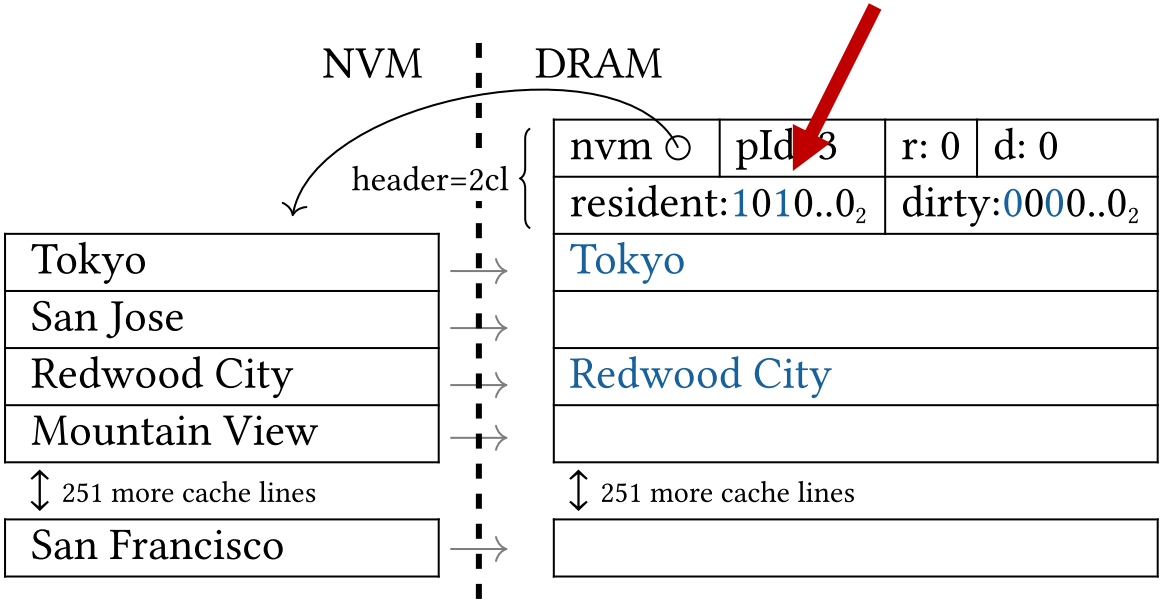
We transfer individual cache lines (64Byte) instead of entire pages (16KB) between DRAM and NVM.

1. Cache-Line-Grained Loading



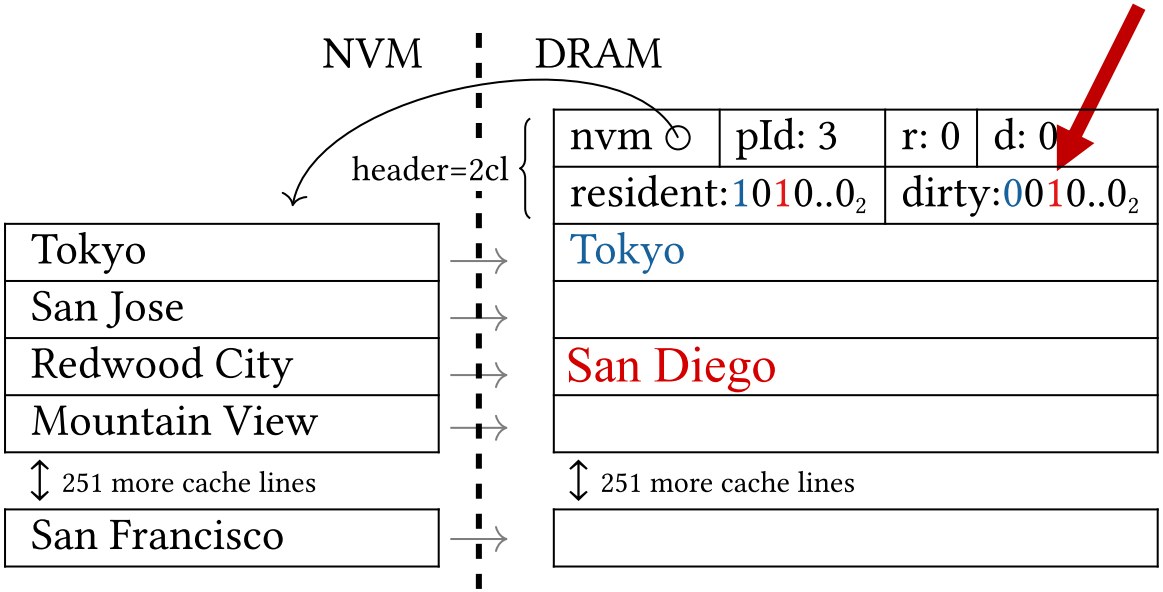
We transfer individual cache lines (64Byte) instead of entire pages (16KB) between DRAM and NVM.

1. Cache-Line-Grained Loading



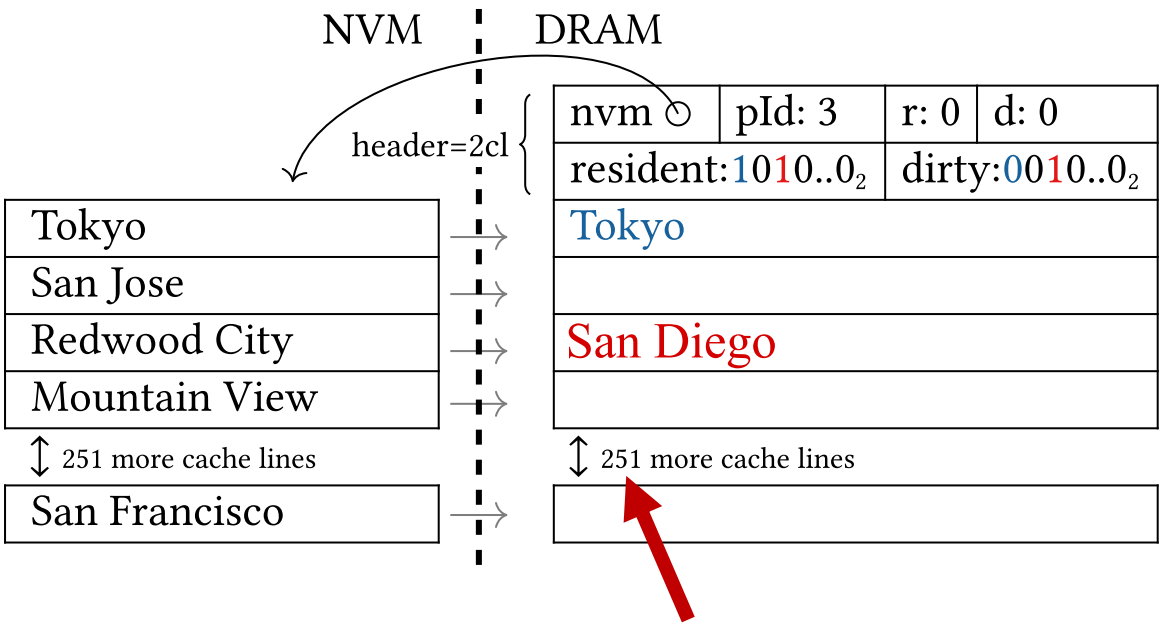
We transfer individual cache lines (64Byte) instead of entire pages (16KB) between DRAM and NVM.

1. Cache-Line-Grained Loading



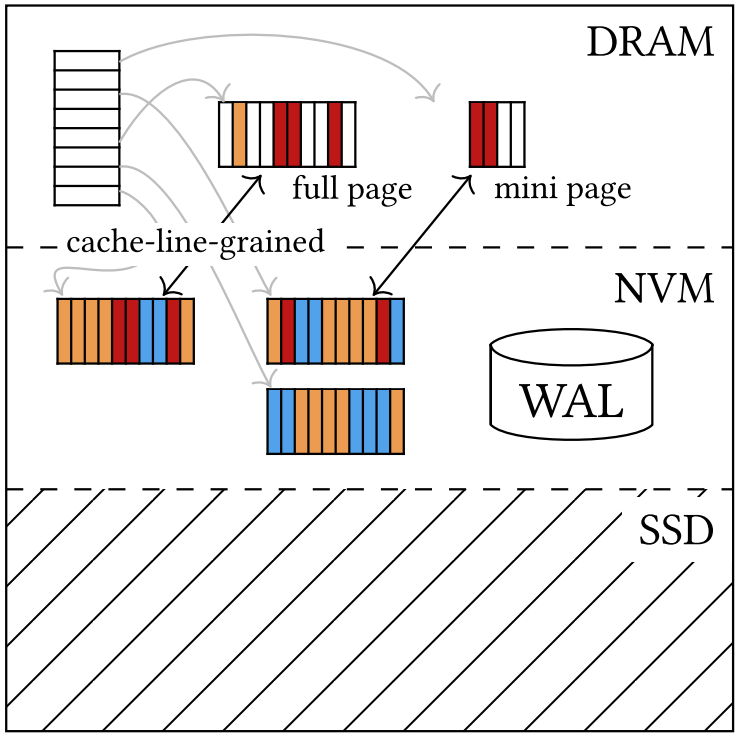
We transfer individual cache lines (64Byte) instead of entire pages (16KB) between DRAM and NVM.

1. Cache-Line-Grained Loading



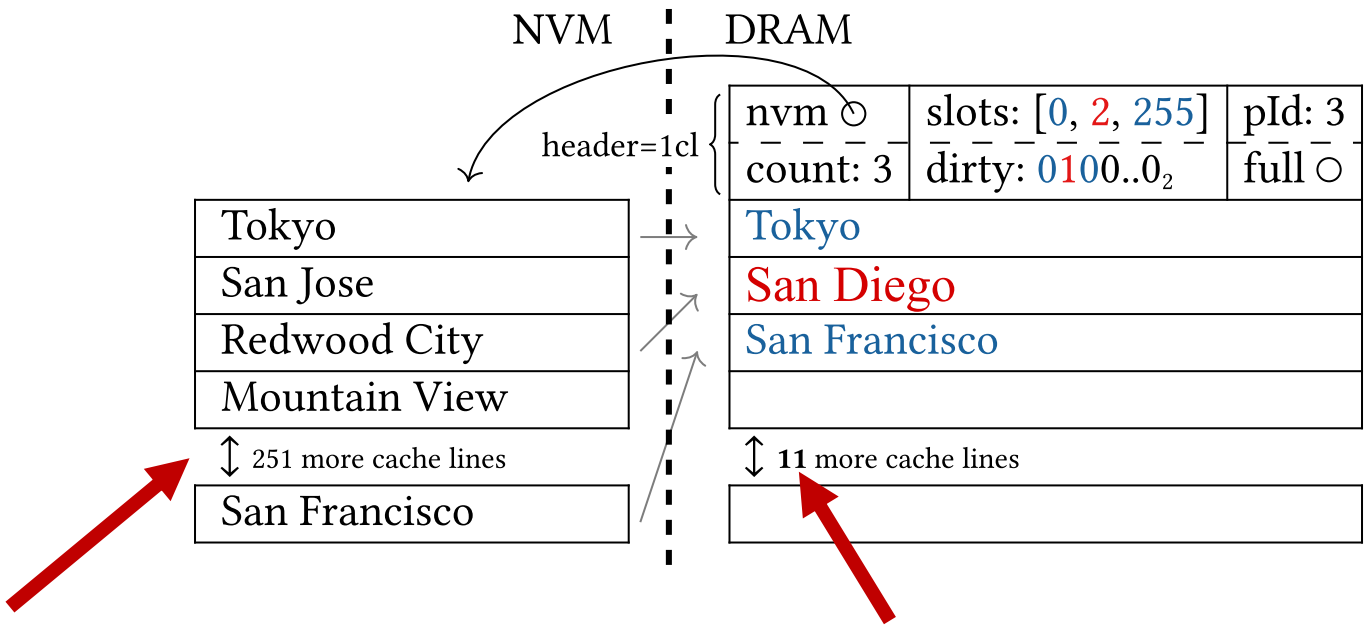
We transfer individual cache lines (64Byte) instead of entire pages (16KB) between DRAM and NVM.

2. Mini Pages



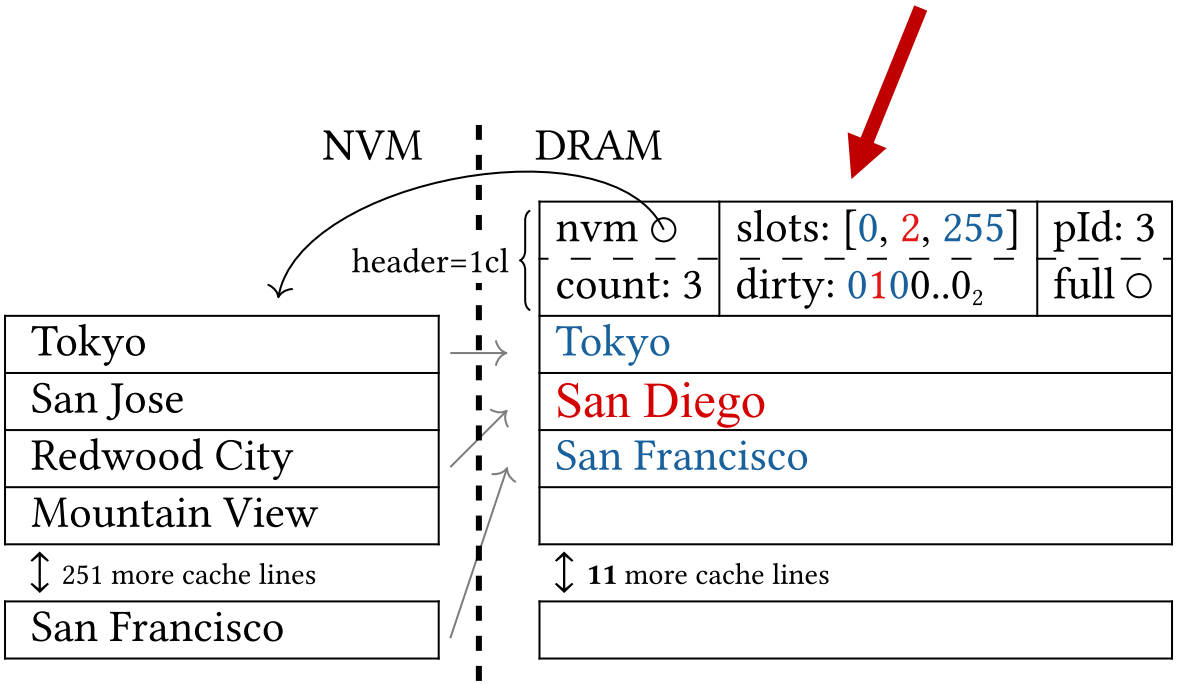
We implement mini pages which store only 16 cache lines (~1KB instead of 16KB).

2. Mini Pages



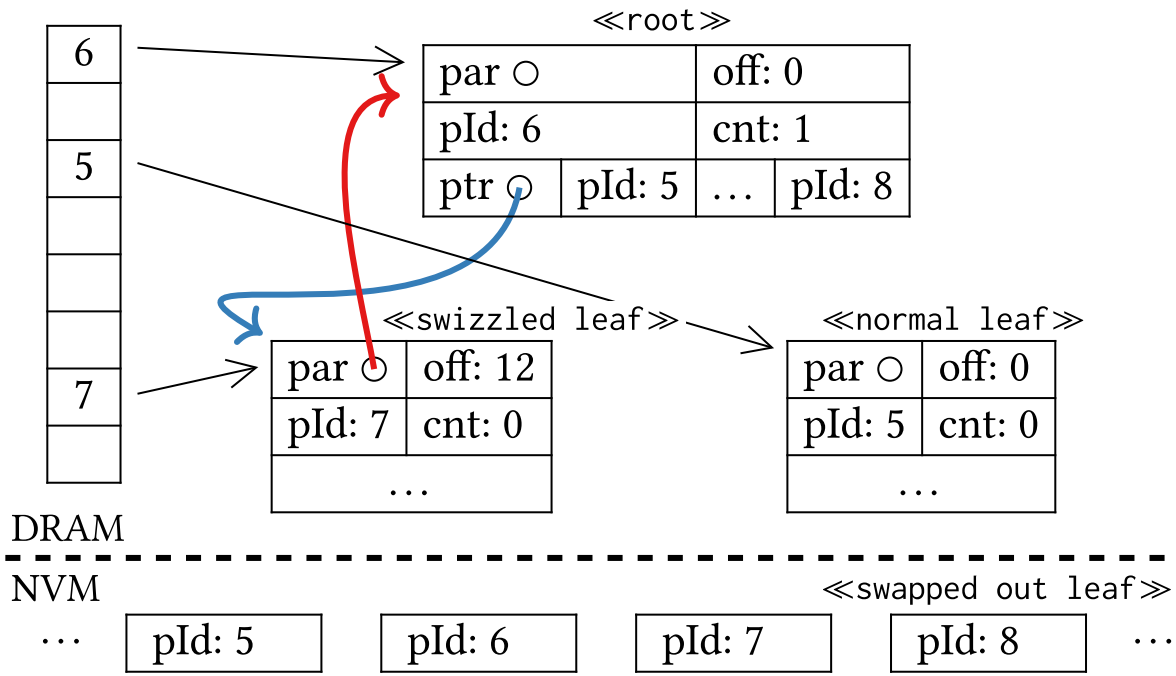
We implement mini pages which store only 16 cache lines (~1KB instead of 16KB).

2. Mini Pages



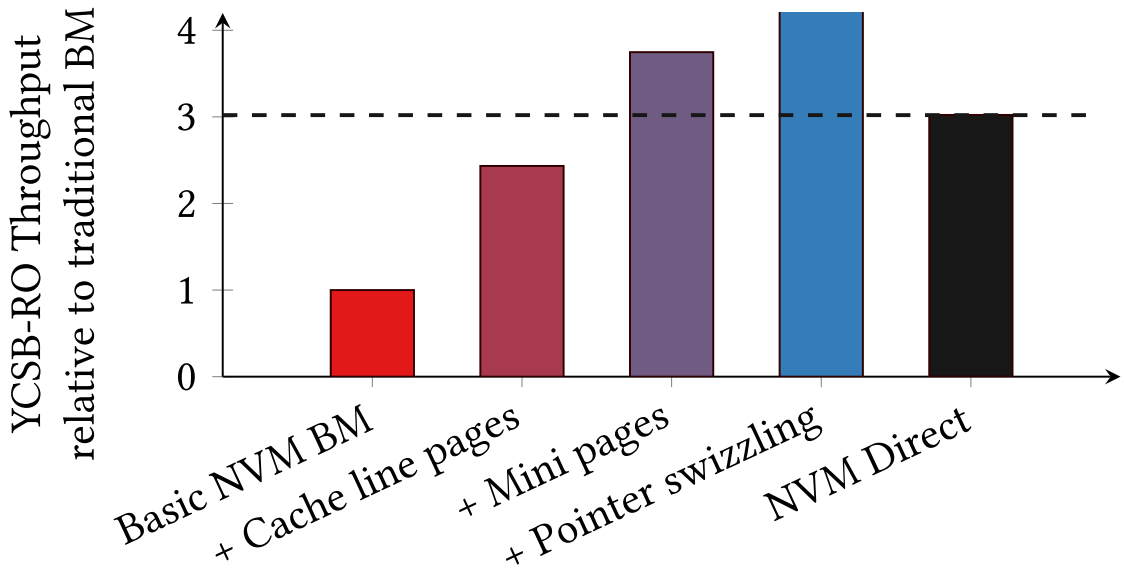
We implement mini pages which store only 16 cache lines (~1KB instead of 16KB).

3. Pointer Swizzling



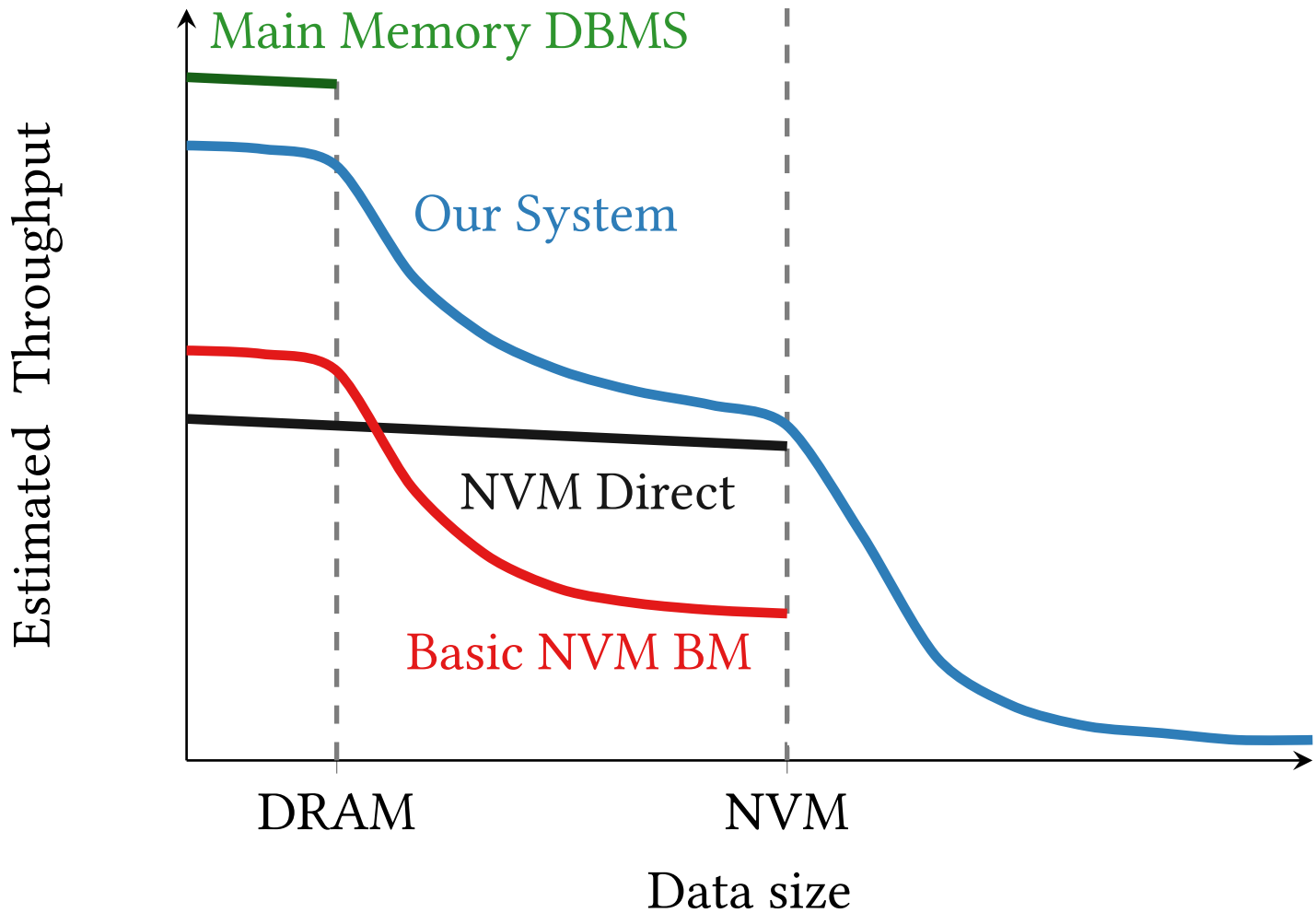
We use pointer swizzling and low-overhead replacement strategies to reduce the buffer manager cost.

Performance Impact of Techniques

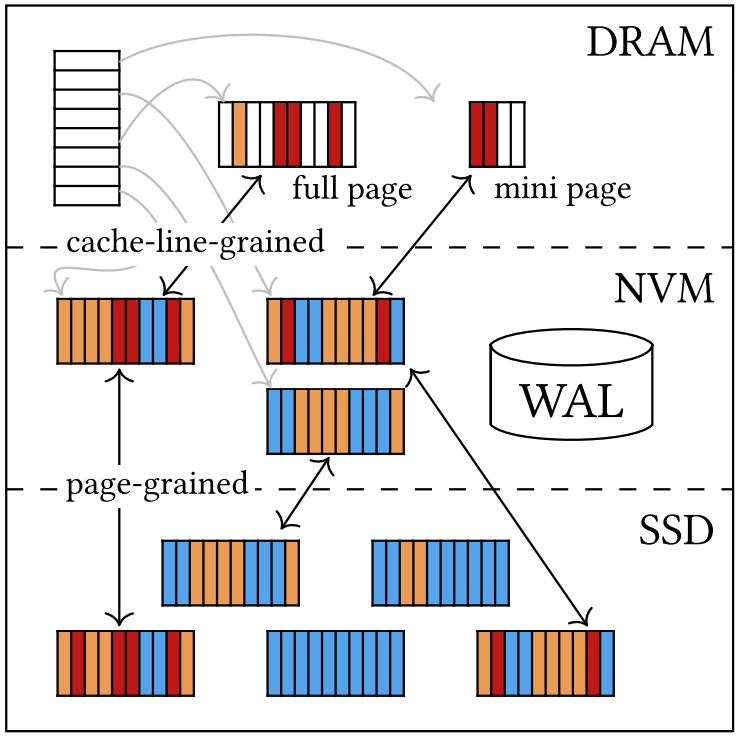


Buffer management needs to be tuned for NVM.

The Ideal System: “Dream Chart”

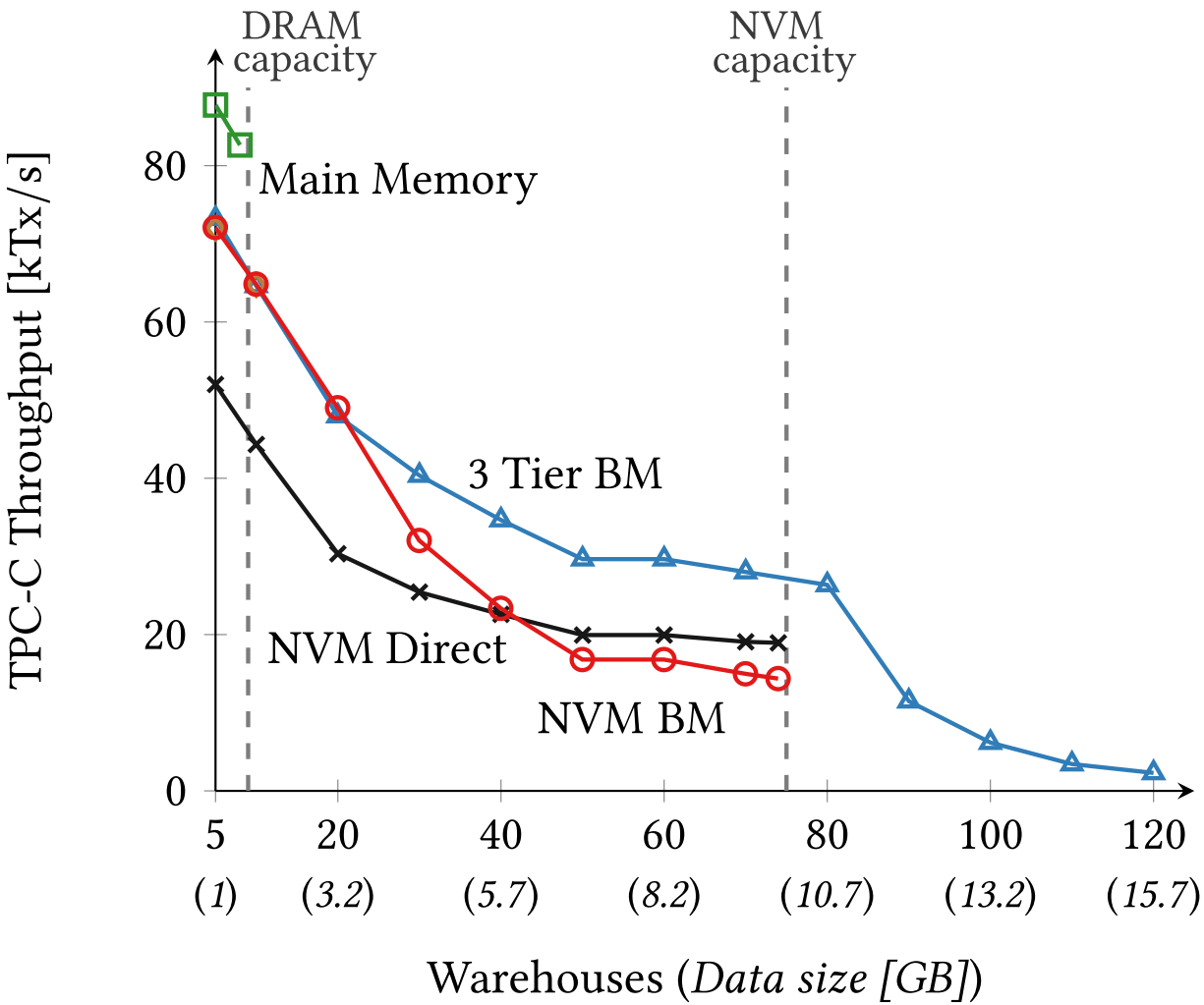


4. Utilize SSDs



By using fixed-size pages, we can extend the maximum possible workload size with SSDs.

Conclusion


DRAM capacity
NVM capacity
Main Memory
3 Tier BM
NVM Direct
NVM BM
TPC-C Throughput [kTx/s]
0
20
40
60
80
5 (1)
20 (3.2)
40 (5.7)
60 (8.2)
80 (10.7)
100 (13.2)
120 (15.7)
Warehouses (Data size [GB])